Python 3.12 AsyncIO & High-Performance Coroutines

Python 3.12.8
VERIFIED RUNTIME

KEY SYNTAX & IDIOMS

Coroutine Definition
```
async def fetch_data(url: str) -> dict:
```
Defines an asynchronous coroutine

Concurrent Gather
```
results = await asyncio.gather(*tasks)
```
Concurrently schedules awaitables

Type Hinting Union
```
def find(query: str | None) -> int:
```
PEP 604 pipe union syntax in 3.10+

Context Manager
```
async with aiohttp.ClientSession() as s:
```
Guarantees safe async socket teardown

COMMON PITFALLS & FIXES

Pitfall: Calling coroutines without await
Rule: Always await coroutine objects or schedule via asyncio.create_task

Pitfall: Blocking sleep in async functions
Rule: Use asyncio.sleep(), never time.sleep(), inside event loops

Pitfall: Ignoring DeprecationWarning
Rule: Run with -W error::DeprecationWarning in CI pipelines

TINY VERIFIED EXAMPLE: ASYNCIO QUICK RUNNER

SOURCE CODE:
```
import asyncio
async def main():
print('Python 3.12 Ready!')
asyncio.run(main())
```

REAL CAPTURED OUTPUT:
```
Python 3.12 Ready!
```